Task 1: Dataset Exploration and Understanding

Students must explore and understand the provided Kaggle dataset before developing their model. Students are required to:

- Describe the image classification problem and the eight target classes.
- Inspect the structure of the training and test data.
- Report the number of images in each class.
- Discuss whether the dataset is balanced or imbalanced.
- Display representative sample images from each class.
- Identify any data quality issues, such as inconsistent image sizes, blurry images, corrupted files, duplicate images, or unclear class boundaries.
- Explain how the dataset will be used for training, validation, and Kaggle submission.

Task 2: Data Preprocessing and Pipeline Preparation

Students must prepare the image data and develop a reproducible preprocessing pipeline. Students are required to:

- Resize images to a suitable input size for the selected model.
- Normalise or rescale pixel values appropriately.
- Encode class labels into a suitable numerical format.
- Split the labelled training data into training and validation sets using a justified strategy.
- Apply appropriate data augmentation techniques where relevant.
- Use suitable data loaders, generators, or dataset objects for efficient training.
- Ensure that preprocessing is applied consistently to training, validation, and test images.
- Avoid data leakage between training, validation, and test data.

Task 3: Model Selection and Pipeline Development

Students must design and implement a complete deep learning image classification pipeline. Students are required to:

- Select and justify a suitable deep learning architecture.
- Use a custom CNN, transfer learning model, Vision Transformer, hybrid CNN--Transformer model, or another appropriate deep learning model.

- Describe the model architecture and its main components.
- Specify the input image size, output layer, activation function, and number of target classes.
- Select and justify the loss function, optimiser, learning rate, batch size, and number of epochs.
- Implement the complete training pipeline in Python.
- Ensure that the pipeline can generate predictions for the Kaggle test set.

Task 4: Training Strategy and Overfitting/Underfitting Management

Students must train their model and monitor its learning behaviour. Students are required to:

- Train the selected model using the prepared training data.
- Monitor training and validation accuracy and loss.
- Identify signs of overfitting or underfitting.
- Apply appropriate strategies to improve generalisation, such as data augmentation, dropout, weight decay, batch normalisation, early stopping, learning-rate scheduling, transfer learning, fine-tuning, or model adjustment.
- Present and discuss learning curves for training and validation performance.
- Justify any changes made to the model or training strategy.

Task 5: Model Evaluation and Kaggle Submission

Students must evaluate their model and submit predictions to the Kaggle competition. Students are required to:

- Evaluate the model on the validation set using suitable classification metrics.
- Report validation accuracy and loss.
- Include additional metrics where appropriate, such as precision, recall, F1-score, confusion matrix, or class-level accuracy.
- Generate predictions for the Kaggle test images.
- Prepare the submission file in the required Kaggle format.
- Submit predictions to the Kaggle competition.
- Report the Kaggle leaderboard score and include evidence of the submission.
- Compare validation performance with Kaggle leaderboard performance.

Task 6: Results Analysis and Improvement Strategies

Students must critically analyse the results and discuss possible improvements. Students are required to:

- Interpret the model performance using validation results and Kaggle score.
- Analyse common misclassifications and difficult classes where possible.
- Discuss possible reasons for performance limitations.
- Evaluate the effectiveness of preprocessing, augmentation, model choice, and training strategy.
- Compare attempted model versions or experimental settings where applicable.
- Propose realistic strategies for improving performance, such as alternative architectures, hyperparameter tuning, improved augmentation, fine-tuning, ensemble methods, or better validation strategies.
- Reflect on the strengths and weaknesses of the final solution.

Task 1: Dataset Exploration & Understanding

1.1 Classification Problem and Target Classes



Figure 1.1.1: Cat photo from Kaggle

This project is an image-classification task, which requires supervised deep learning. A dataset is provided containing testing and training sets with images of both cats and dogs. During training, the model will look at an image and needs to choose one class from the 8 possible classes. For example, the image can be classified as a CatEye or DogEye, but not both at once. The dataset contains images of both cat and dog species. The images can be of the whole animal, or just a facial part like the eye, face, mouth, or nose. With the two animals and 4 facial parts, there are 8 possible classes the model can choose: CatEye, DogEye, CatFace, DogFace, CatMouth, DogMouth, CatNose, DogNose. Because the target class is categorical, this is a multi-class classification problem. The model needs to learn fine-grained visual differences to recognise cats and dogs apart, and then the facial features from each other. The main goal of the model is to achieve a high classification accuracy, which is official evaluation metric that this competition is measuring.

1.2 Dataset Structure

```
Training-label shape: (63660, 2)
Sample-submission shape: (29443, 2)
```

Figure 1.2.1: Image count for train/test sets

The dataset provided to us contains **63,660 training images**, and **29,443 testing images**, for a **total of 93,103 images** in both sets. Four different components were provided being the training images, training label's in csv, testing images, and the sample_submission csv. Both the sample_submission and train_label csv files have two columns being image_id and class. This is because the train_label.csv file maps each image ID to its class for the supervised learning. The sample_submission.csv shows how the submission format should look like displaying the row order and structure.

Component	Rows	Purpose
Training images	63,660	Model training
Testing images	29,443	Kaggle predictions
train_labels.csv	63,660	Training labels
sample_submission	29,443	Submission format

The training images will be fed to the selected model in section 3, where the model will learn to recognise visual patterns and differentiate them. They will then be used to develop the model and for validation using a validation subset of the training set. The testing images have labels, but will be hidden from the model until it creates the final submission. After the submission, kaggle compares the predictions with the hidden labels and returns the accuracy score.

1.3 Class Distribution and Dataset Balance

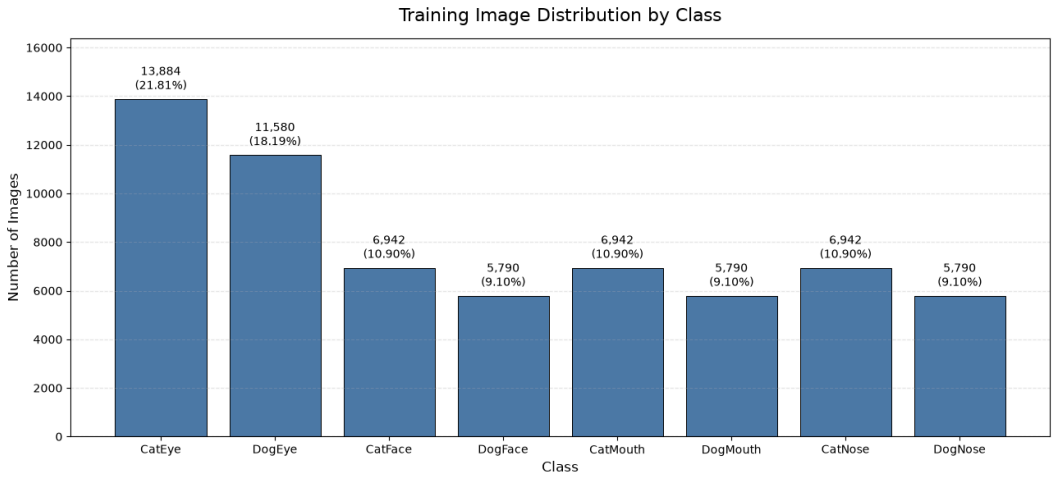


Figure 1.3.1: Distribution of classes

As stated in section 1.1, there are 8 different classes in the provided dataset. This includes two species (cats and dogs) and four facial regions (eye, face, mouth, and nose) which creates a total of 8 different classes. It is important to know the distribution of the classes in the training data, because that is what will be used to train the model. From the distribution in figure 1.3.1, CatEye and DogEye have the highest distribution with 21.81% and 18.19% of the whole training dataset

respectively. Both of these classes contain double the amount of the Face, Mouth, and Nose regions. From figure 1.3.1, the training dataset contains 20% more cat images than dog images. While not completely balanced at 50/50, this indicates there is **no class imbalance** between the two species as 60 to 40 is reasonably balanced. However, the larger classes could influence the model and its overall accuracy more than the minority classes of DogFace, DogMouth, and DogNose. The class-specific precision, recall, and F1-scores should be noted later so that the weaker performance is not hidden by the overall accuracy.

1.4 Representative Images and Visual Variation

There are **8 different classes** in this project's dataset, and while some images are from the same class, they will still be difference from one another. For example, the lighting could be different with one cat in the sun, and the other in the shade. The angle of the photo could also be different, from a close low angle, to a further high angle. Other variables like fur colour, image sharpness, image sizes, and backgrounds also differentiate each image.

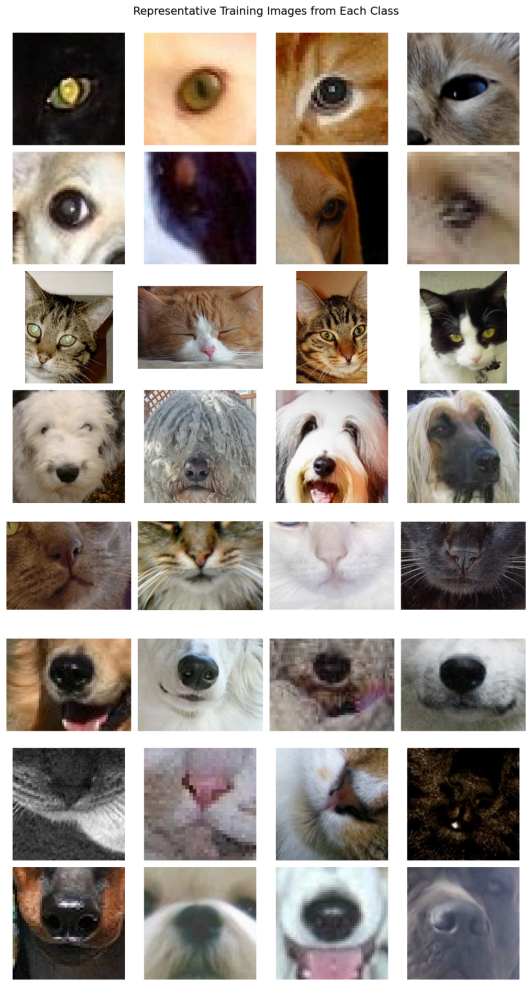


Figure 1.4.1: Representive sample images

There were four images used from every class using a random seed **42,** making the sample selection reproducible on another device or instance. While images from the same class may look different, images from different classes could look completely identical. For example, cats and dogs have similar nose shape and colours, so a photo of both with the same zoom and lighting conditions could be confusing to differentiate for the model. Another problem that could trouble the model is that a cropped image may remove details that help identify if the image is of a cat or dog. These fine-grained differences are what the model has to learn throughout training, so that it can perform well on the testing set, and differentiate between both animals, and their facial regions. After model training,

we can examine what classes were harder to detect, in order to improve the model.

1.5 Image Properties and Data Quality

The dataset provided from kaggle contained a **total of 93,103 images** from all 8 classes in both training and testing sets. All of these images could be opened, and there were **no corrupted or unreadable files**, meaning that no images had to be removed due to corruption. All images were checked if they used grayscale, RGB or both. It was confirmed that all of the **images in both training and testing used RGB** only, thus no conversion was needed.



Figure 1.5.1: Image size distribution graph

The width and height of each image in the training set was examined. It was discovered that there wes a significant number of size combinations, and that images do not use a standard size. The sizes of images ranged from 4x4 to 2,219x2,219, meaning that a fixed input size would need to be selected during data preprocessing. According to part 10, in notebook 1 in the project files, 40x40, 48x48 and 42x42 were the three most common sizes that images had in the dataset. Resizing will be applied to all training, validation, and test images to ensure all images that are fed to the model have a common size.

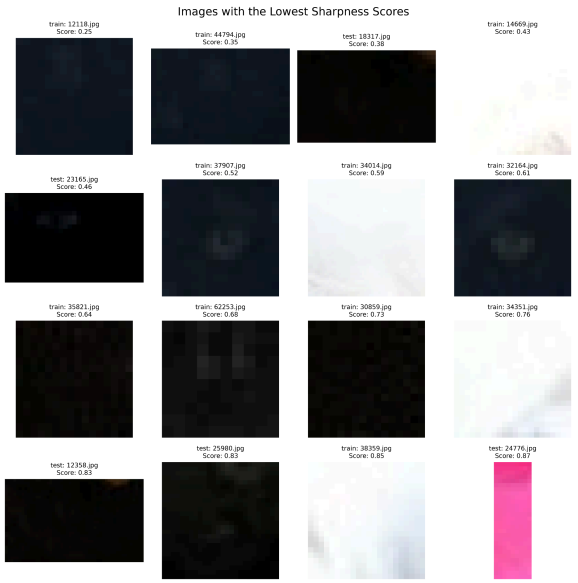


Figure 1.5.2: Low sharpness images

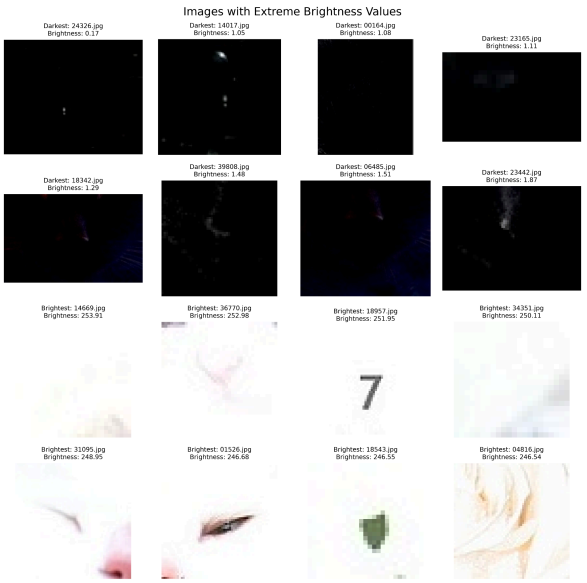


Figure 1.5.3: Extreme brightness images

Figure 1.5.2 shows the images that had the lowest sharpness scores out of all the images in the dataset. Low sharpness images are due to being very dark or light, extremely small width and height, low in detail, and difficult to interpret for the machine. The sharpness score is only used for screen and doesn't prove that an image is just blurry, as it can be due to the reasons mentioned previously. The sharpness score was used to select images that went through manual review. If the images were too low in sharpness, they would need to be considered. However, the sharpness score should not remove any images for the score alone, so no removal was made.

Figure 1.5.3 shows examples of images that are too dark or too light. While extreme brightness images may seem made for model training, it could improve the robustness of the model. This gives the model the chance to learn how to differentiate classes even when the brightness of the image isn't ideal. This will need to be tested, and should not be removed from the training set.



Figure 1.5.4: Duplicate images

A SHA-256 hash was calculated using the contents of images, and if they had the same hash, they were considered exact duplicates. There were 15 duplicates identified in the training set, and 62 in the testing set. There was a total of 77 duplicates in the whole dataset, however none of them was shared between the sets. Duplicates could cause data leakage if one copy is in the training subset

while the other is in the validation subset. Therefore, the duplicate images should be grouped together so that they stay in the same subset, preventing data leaking during training.

1.6 Intended use of the Dataset

The provided dataset is not balanced across all classes, since the eye classes contain double the face, mouth, and nose classes. There is also 20% more images in the cat classes than in the dog classes. This is a moderate imbalance and not extreme imbalance, however this means that the **overall accuracy could be influenced** more to eye and cat classes. This means that while the model could have a strong accuracy, it might not be the same for every class. Because the eye classes are contributing more training samples, they could have a higher influence on predictions, which could decrease the overall accuracy. This will not be confirmed until the results of the model are examined.

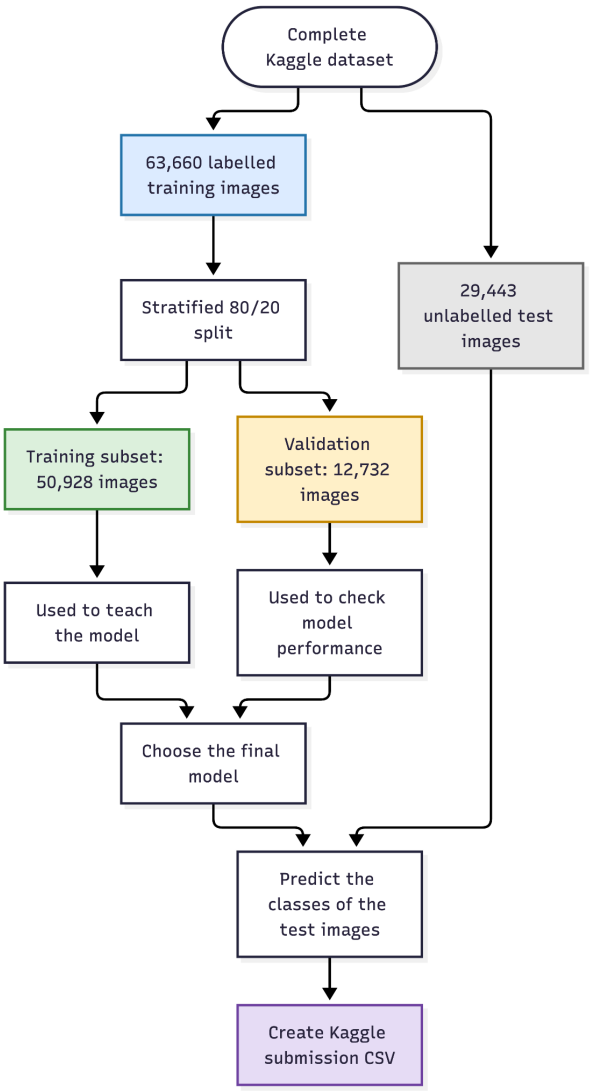


Figure 1.6.1: Flow chart for trianing/validation split

The data will be split into two subsets using the original training data folder. There will be a training subset which will be fed to the model to learn, and a validation subset which the model will try to predict their classes without knowing the target. **Stratification will be used** to split the training dataset while using an **80/20 split**, meaning 80% will be training and 20% on validation. Using stratification ensures that the **original distribution of images in the training data is carried over** to the subsets, making the environment as close to the original as possible.

The 80% training split will be used to update the model's weights, while the 20% validation split measures the performance on the images with the labels not seen. Data augmentation will be

applied while training, which is when new training images are created by flipping, rotating, or brightening existing images. This technique can help the model recognise the 8 classes under different conditions, allowing it to be more robust during image classification. Before the images are fed to the model, they will be resized so that the model works with consistent image sizes and formats. After all predictions have been made, they will be saved in the order that is mentioned in sample_submission.csv.